

Breve spiegazione UML:

---- CONTROLLER -----

GameManagerController: classe che contiene la logica che permette di connettersi ad una partita già esistente o eventualmente crearne una nuova.

GameController: classe che contiene la logica che gestisce l'alternarsi dei turni e la logica di terminazione della partita (per vittoria di uno dei giocatori).

PlayerController: classe che contiene la logica che permette ad un singolo giocatore di effettuare le mosse prelevando le tessere dalla LivingRoom (oggetto singolo condiviso tra i vari playerController) ed inserendole nella propria shelf (oggetto proprietario del singolo playerController). Contiene inoltre la logica di routine che controlla, dopo aver effettuato la mossa, il raggiungimento di personal e common goals. Contiene infine la logica che permette di valutare la shelf in base alla presenza o meno di gruppi di tessere uguali adiacenti.

ChatController: classe che contiene la logica che permette di inviare un messaggio.

---- MODEL -----

GamesManager: Interfaccia della classe che si occupa di memorizzare e rendere accessibili i match (Funzionalità aggiuntiva multi-partita).

StandardGamesManager: implementazione di GamesManager che sfrutta una Map per la gestione dei match

Game: Classe che rappresenta l'intera struttura del gioco (Giocatori, LivingRoom, CommonGoals e la Chat)

LivingRoom: Interfaccia di classe che si occupa di memorizzare lo stato della livingRoom e che contiene l'algoritmo (un pattern) di riempimento della board.

StandardLivingRoom: implementa l'interfaccia secondo il modello di board stabilito dalle regole del gioco, contiene l'attributo **Bag** rappresentante il sacchetto dal quale vengono pescate le tessere necessarie al riempimento della board.

CommonGoal: Classe astratta che rappresenta un commonGoal, si è stabilita la rappresentazione della pila di token come uno stack riempito alla generazione dell'oggetto, i valori caricati nella pila di token e l'ordine con il quale ciò avviene è invece affidato all'estensione della classe.

I **Token** sono rappresentati da enumerabili il cui valore è rappresentato da un attributo di tipo intero.

L'algoritmo che definisce il raggiungimento del commonGoal è affidato alle implementazioni dell'interfaccia **GoalEvaluator**, tali implementazioni definiscono anche la descrizione del pattern richiesto.

Il caso standard prevede 12 algoritmi ciascuno rappresentato da una propria classe.

Player: Classe che rappresenta il "giocatore", contiene:

- Shelf in forma di matrice N*N
- Punti guadagnati dai token (CommonGoals) in forma di numero intero
- Riferimenti ai propri personalGoals
- Uno stato che identifica quand'è il turno del giocatore, quando è in attesa e quando ha terminato la propria shelf (lo stato serve a permettere l'alternanza dei turni)

PersonalGoal: interfaccia della classe che definisce l'algoritmo secondo il quale un giocatore ha raggiunto o meno un personalGoal. La definizione di personalGoal si discosta dalla definizione delle regole del gioco che vede un *unico pattern* di 6 tessere diverse in 6 posizioni: nel caso standard un giocatore avrà **6 personalGoal ognuno dei quali richiede la posizione di un tipo particolare di tessera in una posizione particolare della shelf**. Ciò consente di espandere il concetto di personalGoal ad eventuali diverse interpretazioni ed implementazioni future. Sarà poi compito del PlayerController alla fine della partita restituire i punti guadagnati dal conseguimento di n personalGoal.

Chat: interfaccia della classe che si occupa di memorizzare e rendere accessibili i messaggi inviati dai giocatori, nell'implementazione **StandardChat** un messaggio è dichiarato inviato ad "everyone" se il suo IdDestination è una String vuota.

Message: interfaccia della classe rappresentante un messaggio.

Tile: oggetti di tipo enumerabile, Tile dello stesso tipo sono accomunate uguale valore dell'attributo **color**
